

Práctica Tema 3

Serafi Nebot Ginard

27/03/2025

Contents

1	Sistema utilizado	2
1.1	Configuración del sistema	2
1.2	Instalación Sysbench	2
1.2.1	Instalar dependencias	2
1.2.2	Descarga código fuente	2
1.2.3	Actualizar la versión de config.guess y config.sub para que ./configure funcione	2
1.2.4	Compilar a partir del código fuente	2
1.2.5	Instalar el ejecutable generado en el sistema	2
2	Implementación del experimento	3
2.1	Carga	3
2.2	CPU	3
2.3	RAM	4
2.4	Automatización	4
A	Test appendix	5

1 Sistema utilizado

- Macbook Air 2022, CPU Apple M2 8-core, RAM 16GB
- VMWare Fusion 13.6.3
 - Ubuntu 22.04.5 LTS
 - 30GB disco
 - 12GB RAM (máximo recomendado por VMWare Fusion)
 - 8 cores
 - Sysbench 0.4.12

1.1 Configuración del sistema

El ordenador en el que he realizado toda la práctica es un Macbook Air 2022 con CPU Apple M2 de 8 núcleos y 16GB de RAM. He utilizado VMWare Fusion 13.6.3 para crear una máquina virtual con 30GB de disco, 12GB de RAM y 8 núcleos de CPU.

Debido a que me he encontrado con muchos problemas para instalar Ubuntu 16.04, la versión preferible para realizar esta práctica porque se puede instalar una versión de sysbench < 1.0 directamente, he decidido instalar Ubuntu 22.04.5 y instalar una versión de sysbench < 1.0 manualmente.

1.2 Instalación Sysbench

Debido a que no he encontrado ninguna versión de Sysbench < 1.0 precompilada para Ubuntu 22.04.5, he descargado el código fuente de la versión 0.4.12, que es la más reciente antes de la 1.0, del [repositorio de paquetes de Fedora](#) y he seguido los siguientes pasos.

1.2.1 Instalar dependencias

```
sudo apt install make automake libtool pkg-config libaio-dev autoconf libtool-bin libtool
```

1.2.2 Descarga código fuente

```
mkdir install
cd install
wget https://src.fedoraproject.org/repo/pkgs/sysbench/sysbench-0.4.12.tar.gz/3a6d54fdd3fe002328e445820639
tar -xvf sysbench-0.4.12.tar.gz
cd sysbench-0.4.12
```

1.2.3 Actualizar la versión de config.guess y config.sub para que ./configure funcione

```
wget -O config/config.guess 'https://git.savannah.gnu.org/gitweb/?p=config.git;a=blob_plain;f=config.guess'
wget -O config/config.sub 'https://git.savannah.gnu.org/gitweb/?p=config.git;a=blob_plain;f=config.sub'
```

1.2.4 Compilar a partir del código fuente

```
./autogen.sh
autoupdate
./configure --without-mysql # se omite mysql para evitar instalar dependencias; no se necesita para los tests
sudo make LIBTOOL=/usr/bin/libtool
sudo make LIBTOOL=/usr/bin/libtool install
```

1.2.5 Instalar el ejecutable generado en el sistema

```
sudo cp sysbench/sysbench /usr/local/bin/
sudo chmod +x /usr/local/bin/sysbench
sysbench --version
```

2 Implementación del experimento

2.1 Carga

Para aplicar una carga al sistema se ha utilizado el test de CPU de cálculo de numeros primos del sysbench. Por ejemplo, para cargar la CPU al 75% ($8\text{ CPUs} \cdot 0.75 = 6\text{ CPUs}$) con el test de 100000 numeros primos:

```
$ sysbench --num-threads=6 --test=cpu --cpu-max-prime=100000 run
```

```
sysbench 0.4.12: multi-threaded system evaluation benchmark
```

Running the test with following options:

Number of threads: 6

Doing CPU performance benchmark

Threads started!

Done.

Maximum prime number checked in CPU test: 100000

Test execution summary:

total time:	4.7592s
total number of events:	10000
total time taken by event execution:	28.5390
per-request statistics:	
min:	2.03ms
avg:	2.85ms
max:	72.85ms
approx. 95 percentile:	7.98ms

Threads fairness:

events (avg/stddev):	1666.6667/35.03
execution time (avg/stddev):	4.7565/0.00

Como el sysbench ya nos proporciona el tiempo de respuesta (campo `total time`; primera línea del resumen de ejecución) no se necesita monitorizar el tiempo de ejecución del sysbench.

2.2 CPU

Para monitorizar el porcentaje de uso de la CPU se ha utilizado el la primera línea del archivo `/proc/stat`:

```
$ cat /proc/stat | head -n 1
cpu 12070 86 2074 516794 445 0 40 0 0 0`
```

Que indica la cantidad de *jiffies* de CPU desde que se ha encendido el sistema con el siguiente formato: `cpu [user] [nice] [system] [idle] [iowait] [irq] [softirq] [steal] [guest] [guest_nice]`. Esto significa que el “tiempo” total de CPU es la suma de todos los campos, “tiempo” que la CPU ha estado sin hacer nada es la suma de los campos `idle` y `iowait`, y la utilización real de la CPU es la resta entre el tiempo total y el tiempo sin hacer nada.

Debido a que estos campos representan el acumulado hasta el momento, se debe calcular la diferencia de cada uno de los campos para cada intervalo de monitorización. De tal forma, se puede calcular el porcentaje de utilización total de la CPU:

$$U_{CPU} = \frac{(T_i - T_{i-1}) - (I_i - I_{i-1})}{T_i - T_{i-1}} \cdot 100 = 1 - \frac{I_i - I_{i-1}}{T_i - T_{i-1}} \cdot 100$$

siendo T_i la suma de todos los campos y I_i la suma de los campos `idle` y `iowait`, en el instante i .

2.3 RAM

Para monitorizar la utilización de la memoria RAM se han utilizado las dos primeras líneas del archivo `/proc/meminfo`:

```
$ cat /proc/meminfo | head -n 2
MemTotal:      12227312 kB
MemFree:       10558068 kB
```

Que indican la memoria total del sistema y la memoria libre, respectivamente, en kilobytes ($1KB = 2^{10}B$). A diferencia del archivo `/proc/stat`, son valores absolutos y no es necesario calcular las diferencias entre intervalos de monitorización. Se puede calcular el porcentaje de utilización directamente:

$$U_{RAM} = \frac{M_{TOTAL} - M_{FREE}}{M_{TOTAL}} \cdot 100$$

siendo M_{TOTAL} la memoria total del sistema y M_{FREE} la memoria libre.

2.4 Automatización

Para automatizar el proceso de ejecutar una carga específica, lanzar los monitores, pararlos cuando la carga termine, y recuperar todos los datos generados se ha hecho con varias funciones de Python.

```
from __future__ import annotations

import re
import time
import subprocess
from pathlib import Path

TIME_UNITS = { "h": 60*60, "m": 60, "s": 1, "ms": 1/1000, "us": 1/1000000 }

def cpu() -> tuple[float, float]:
    with Path("/proc/stat").open("r") as f:
        # field order: [user] [nice] [system] [idle] [iowait] [irq] [softirq] [steal] [guest] [guest_nice]
        fields = list(map(int, f.readline().strip().split()[1:]))
        return sum(fields), fields[3] + fields[4]

def mem() -> tuple[float, float]:
    with Path("/proc/meminfo").open("r") as f:
        return float(f.readline().split()[1].split()[0]) * 1024, float(f.readline().split()[1].split()[0]) *

def load(*, num_threads: int, max_prime: int, monitor_interval: float, print_overhead: int = False) -> tuple[
    cmd = ["sysbench", f"--num-threads={num_threads}", "--test=cpu", f"--cpu-max-prime={max_prime}", "run"]
    cpu_prev, time_start = cpu(), time.time()
    proc = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE) # noqa: S603

    cpu_usage, mem_usage = [], []
    while True:
        time_total = time.time() - time_start
        sleep_time = monitor_interval - time_total
        if print_overhead: print(f"monitor time: {time_total:.6f}; monitor interval: {monitor_interval:.6f};")
        time.sleep(sleep_time)
        if proc.poll() is not None: break
        time_start = time.time()

        cpu_current = cpu()
        total_diff, idle_diff = (a - b for a, b in zip(cpu_current, cpu_prev))
        cpu_usage.append((1 - idle_diff / total_diff) * 100 if total_diff > 0 else 0)
        cpu_prev = cpu_current
```

```
mem_total, mem_free = mem()
mem_usage.append((1 - mem_free / mem_total) * 100)

stdout, *_ = proc.communicate()
ptrn = re.compile(r"\s*(total time:)\s*\b(\d+\.\d+)(h|m|s|ms|us)\b")
load_time = next(float(p.group(2)) * TIME_UNITS[p.group(3)] for l in stdout.splitlines() if (p := ptrn

return load_time, cpu_usage, mem_usage, stdout
```

A Test appendix